



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE, INDIA

Subject: Computer Organisation and Design

MCA134

MISSION

CHRIST is a nurturing ground for an individual's holistic development to make effective contribution to

VISION

Excellence and Service

CORE VALUES

Faith in God | Moral Uprightness
Love of Fellow Beings

Unit 1

Basics of digital electronics and Micro operations

Subject: Computer Organisation and Design

MISSION

CHRIST is a nurturing ground for an individual's holistic development to make effective contribution to

VISION

Excellence and Service

CORE VALUES

Faith in God | Moral Uprightness
Love of Fellow Beings

COURSE OUTCOMES

CO1: Understand and analyze computer architecture and organization, computer arithmetic, and CPU design

CO2: Compare the design issues in terms of speed, technology, cost and performance.

CO3: Identify the performance of various classes of Memories, build large memories using small memories for better performance and analyze arithmetic for ALU implementation

Unit I (9HRS)

BASICS OF DIGITAL ELECTRONICS AND MICRO OPERATIONS:

Basics Of Digital Electronics: Multiplexers and De multiplexers, Decoder and Encoder, Registers., shift registers, Introduction to combinational circuit, introduction to sequential Circuits Register Transfer and Micro Operations: Register Transfer Language and Register Transfer, Bus and Memory Transfer, Logic Micro Operations, Shift Micro Operations, Design of arithmetic logic unit., arithmetic microoperations

Unit 2(9HRS)

COMPUTER ARITHMETIC: Data representation: signed number representation, fixed and floating point representations, character representation. Computer arithmetic - integer addition and subtraction, ripple carry adder, carry look-ahead adder, etc. multiplication - shift-and-add, Booth multiplier, carry save multiplier, etc. Division - non-restoring and restoring techniques, floating point arithmetic.

Unit III(9HRS)

BASIC PROCESSING MODULE: Fundamental concepts – Execution of a complete instruction – Multiple bus organization – Hardwired control – Micro programmed control -Basic concepts – Data hazards – Instruction hazards – Influence on Instruction sets – Data path and control consideration – Superscalar operation.

Unit IV (9HRS)

MEMORY SYSTEM: Memory Hierarchy and Processor Vs Memory

Speed– Semiconductor RAMs – ROMs –

Speed – size and cost – Cache memories – Performance consideration –

Virtual memory-

Memory Management requirements – Secondary storage.

UNIT-V (9HRS)

PARALLEL PROCESSING: Introduction to Parallel Processing: Pipelining, Characteristics of multiprocessors, Interconnection Structures, parallel processing Latest technology and trends in computer architecture : multi-cores processor., next generation processors architecture, microarchitecture, latest processor for smartphone or tablet and desktop Multiprocessors : Categorization of multiprocessors(SISD,MIMD,SIMD.SPMD), Introduction to GPU

Links for simulation

<https://academo.org/demos/logic-gate-simulator/>

<https://circuitverse.org/simulator>

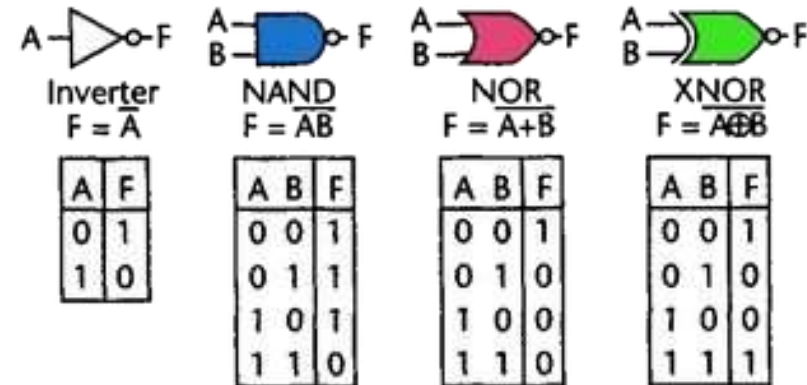
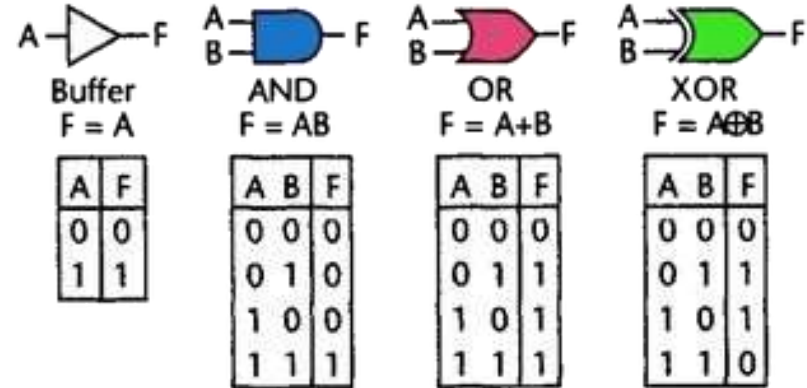
https://www.youtube.com/watch?v=cNN_tTXABUA

Digital circuits

A circuit implemented using a number of logic gate and takes the input in the binary form (0's and 1's) is called as Digital Circuit.

• Types

- Combinational digital circuits
- Sequential digital circuits



COMBINATIONAL LOGIC CIRCUITS

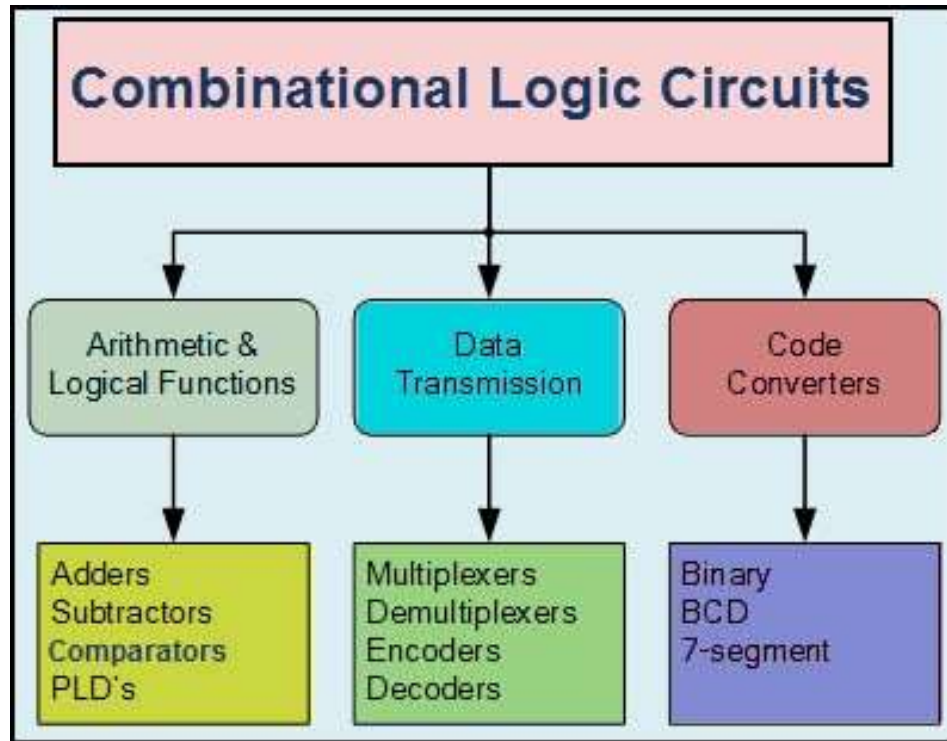
- **Combinational Logic Circuits** are made up from basic gates (AND, OR, NOT) or universal gates (NAND, NOR) gates that are "combined" or connected together to produce more complicated switching circuits.
- logic gates are the building blocks of combinational logic circuits.
- “the **outputs** at any instant of time depends on the inputs **present** at that instant only.”
- For the design of Combinational digital circuits Basic gates (AND, OR, NOT) or universal gates (NAND, NOR) are used.
- Examples for combinational digital circuits are Half adder, Full adder, Half subtractor, Full subtractor, Code converter, Decoder, Multiplexer, Demultiplexer, Encoder, ROM, etc.

Combinational Circuits (cont.)

1. A truth table that lists the output values for each combination of the input variables, or
2. m Boolean functions, one for each output variable.



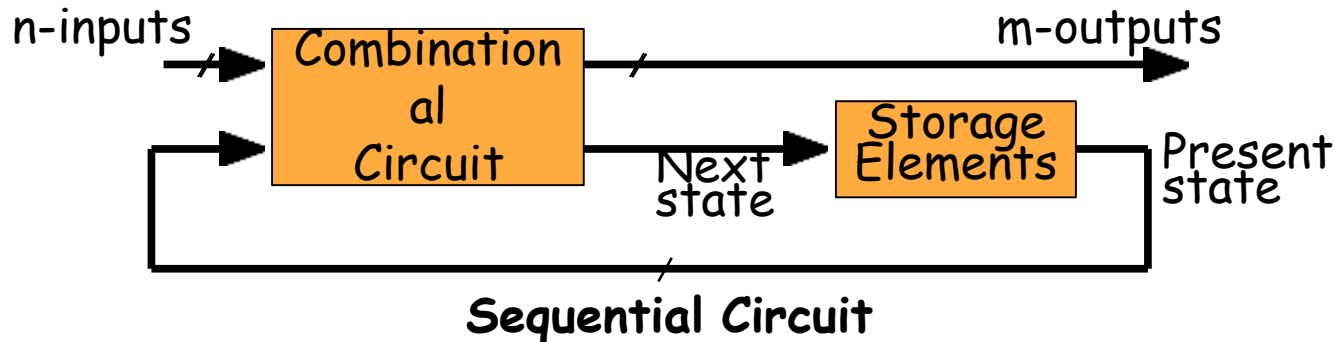
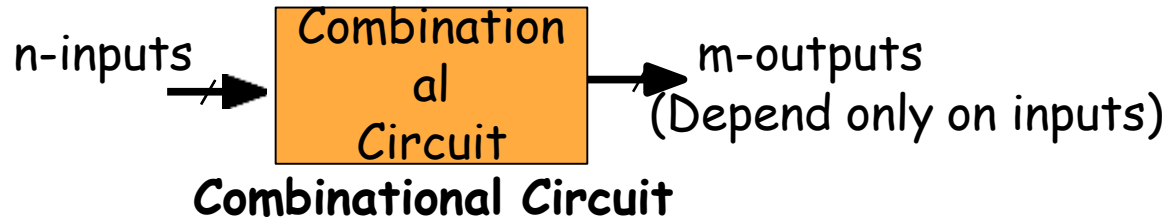
CLASSIFICATION OF COMBINATIONAL LOGIC CIRCUIT



Combinational vs. Sequential Circuits

- Combinational circuits are memory-less. Thus, the output value depends ONLY on the current input values.
- Sequential circuits consist of combinational logic as well as memory elements (used to store certain circuit states). Outputs depend on BOTH current input values and previous input values (kept in the storage elements).

Combinational vs. Sequential Circuits

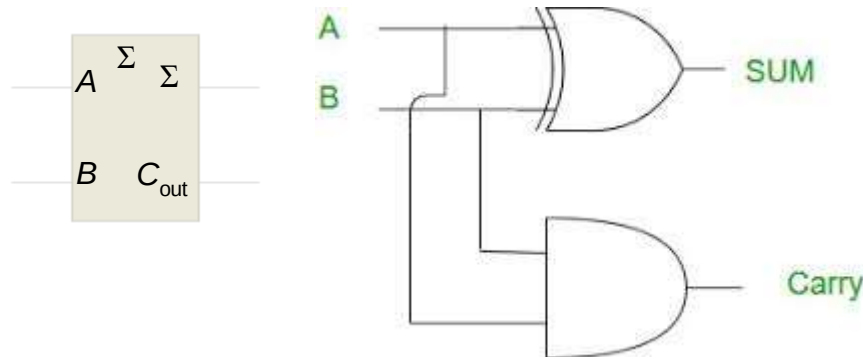


Half-Adder

Basic rules of binary addition are performed by a **half adder**, which has two binary inputs (A and B) and two binary outputs (Carry out and Sum).

Inputs		Outputs	
A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

The logic symbol and equivalent circuit are:



For Carry

		B	
		0	1
A	0	0	0
	1	0	1

$$\text{Carry} = AB$$

For Sum

		B	
		0	1
A	0	0	1
	1	1	0

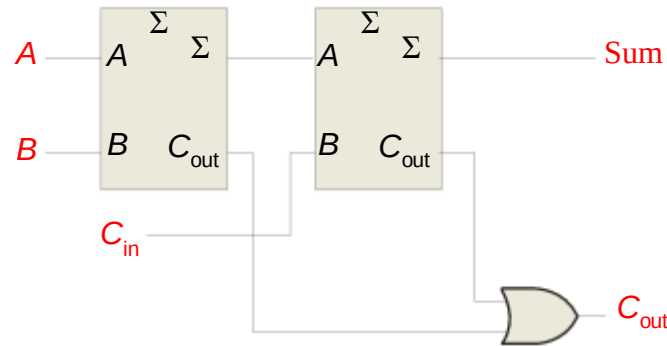
$$\begin{aligned}\text{Sum} &= A\bar{B} + \bar{A}B \\ &= A \oplus B\end{aligned}$$

Fig. 3.12 Maps for half-adder

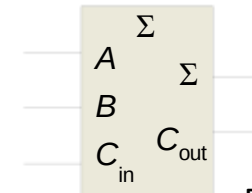
Full-Adder

full adder has three binary inputs (A , B , and Carry in) and two binary outputs (Carry out and Sum).

A full-adder can be constructed from two half adders as shown:



Inputs			Outputs	
A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Boolean
Equation-----
Symbol

For Carry (C_{out})

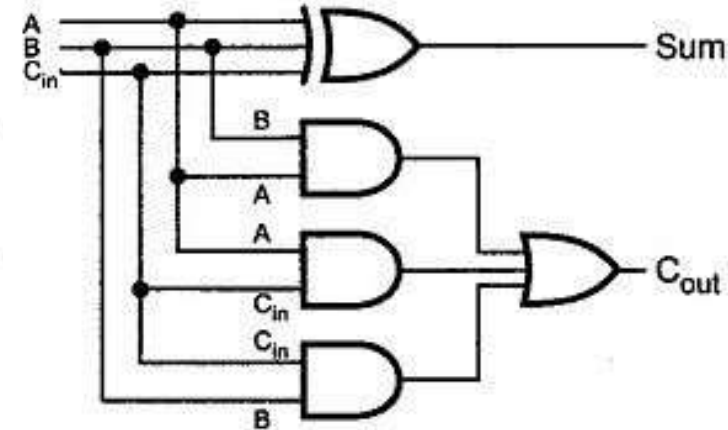
A \ BC_{in}				
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$C_{out} = AB + AC_{in} + BC_{in}$$

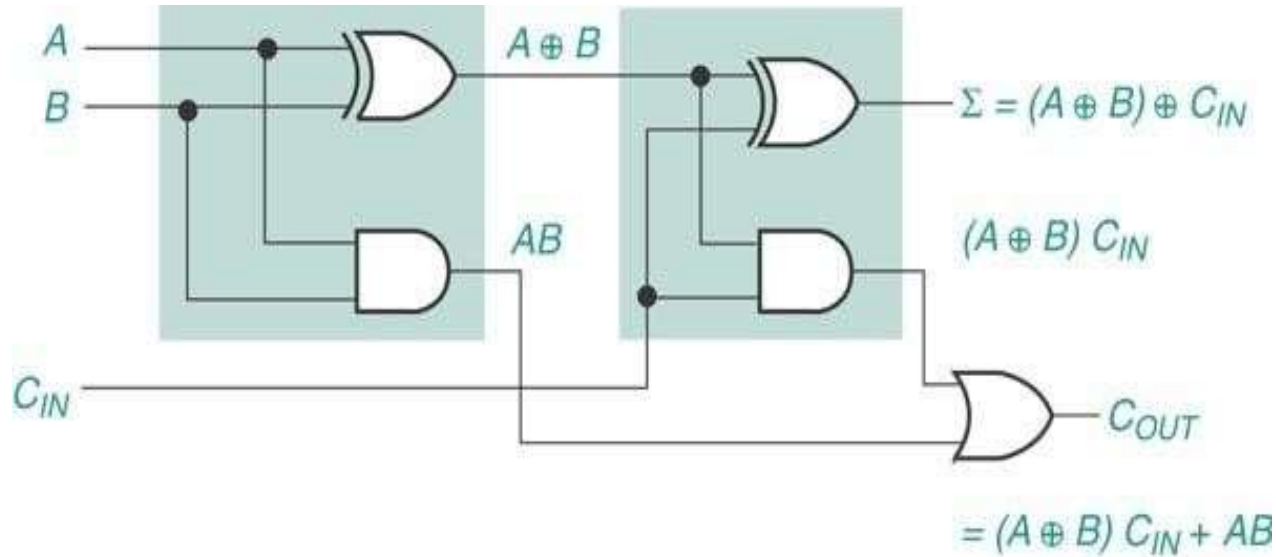
For Sum

A \ BC_{in}				
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

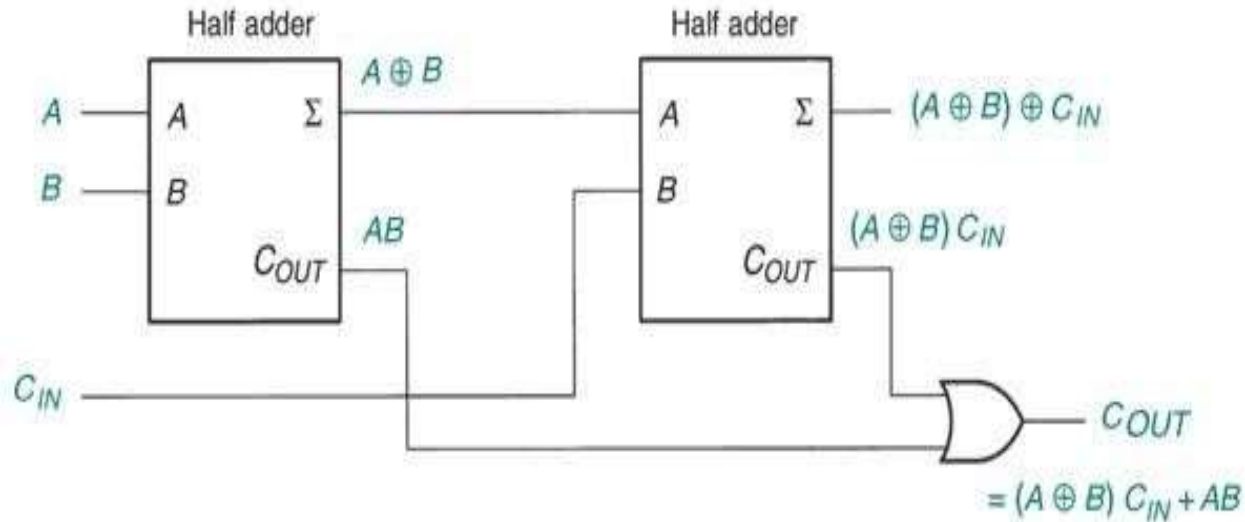
$$\text{Sum} = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$



Full Adder as two half adders



Full Adder as 2 HAs



HALF SUBTRACTOR

- The half-subtractor is a combinational circuit which is used to perform subtraction of two bits.
- It has two inputs, A (minuend) and B (subtrahend) and two outputs Difference and Borrow.
- Half adder can be converted into half subtractor with an additional inverter.

	A	B
	$\bar{A}$	$\bar{B}$
	0	1
	A	B
	1	0

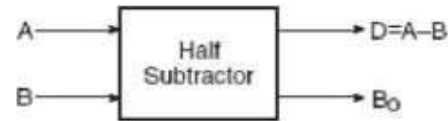
$$\Sigma = A \oplus B$$

	A	B
	$\bar{A}$	$\bar{B}$
	0	1
	A	B
	0	0

$$B_{out} = \bar{A} \cdot B$$

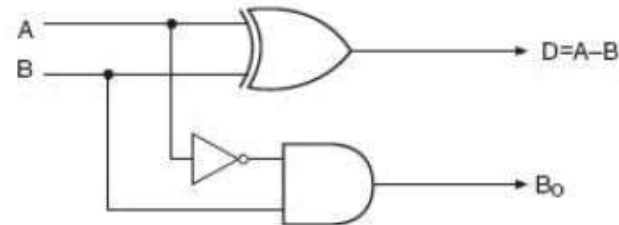
$$D = \bar{A} \cdot B + A \cdot \bar{B}$$

$$B_o = \bar{A} \cdot B$$

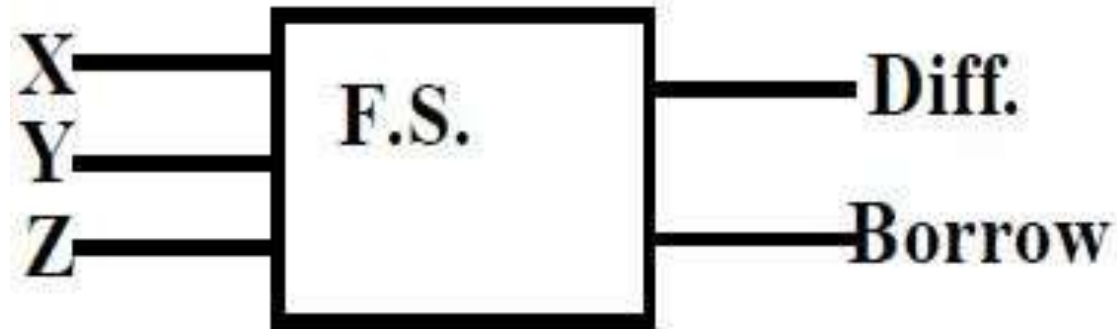


A	B	D	B _o
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Half Subtractor



BLOCK DIAGRAM



Full SUBTRACTOR

- **Full subtractor:** It subtracts one bit from the other by taking pervious borrow into account and generates difference and borrow
- Full adder can be converted into full subtractor with an additional inverter.

Inputs			Outputs	
A	B	B _{in}	D	B _{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Circuit diagram for full subtractor

For D

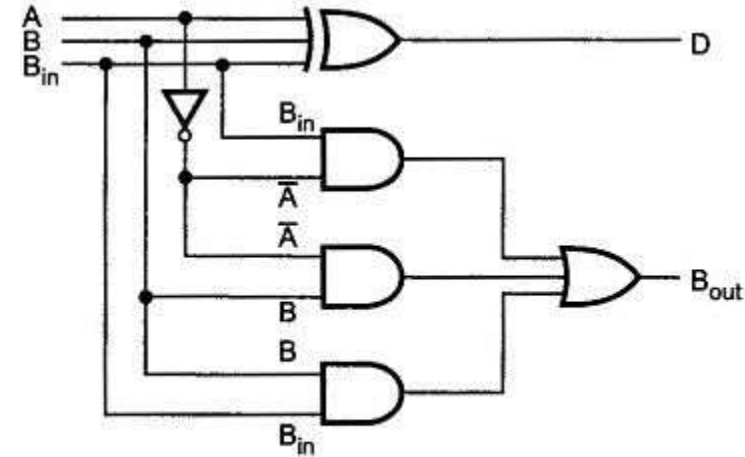
BB _{in}	00	01	11	10
A				
0	0	1	0	1
1	1	0	1	0

$$D = \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + A\bar{B}\bar{B}_{in} + AB B_{in}$$

For B_{out}

BB _{in}	00	01	11	10
A				
0	0	1	1	1
1	0	0	1	0

$$B_{out} = \bar{A}B_{in} + \bar{A}B + BB_{in}$$



Multiplexer (MUX)?

(MUX) function is to select one of several input signals and passes it on to the output.

The select lines determine which input is connected to the output.

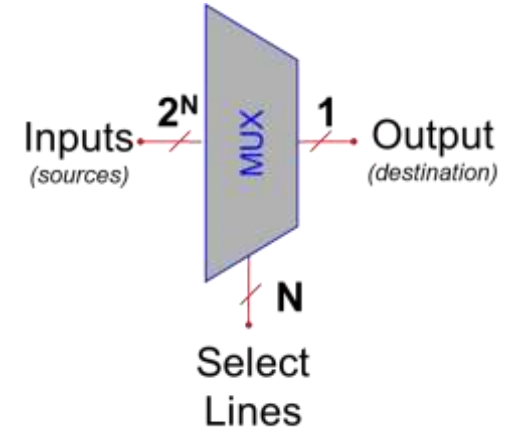
MUX Types

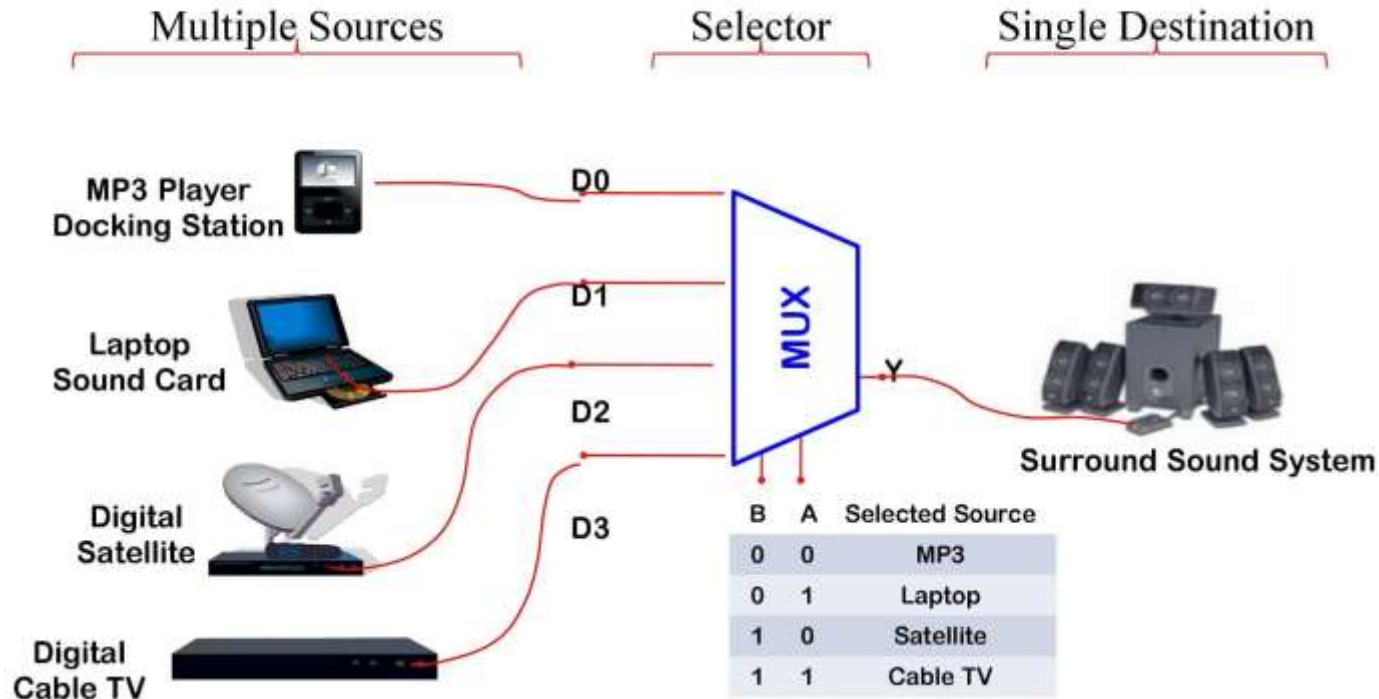
2-to-1 (1 select line)

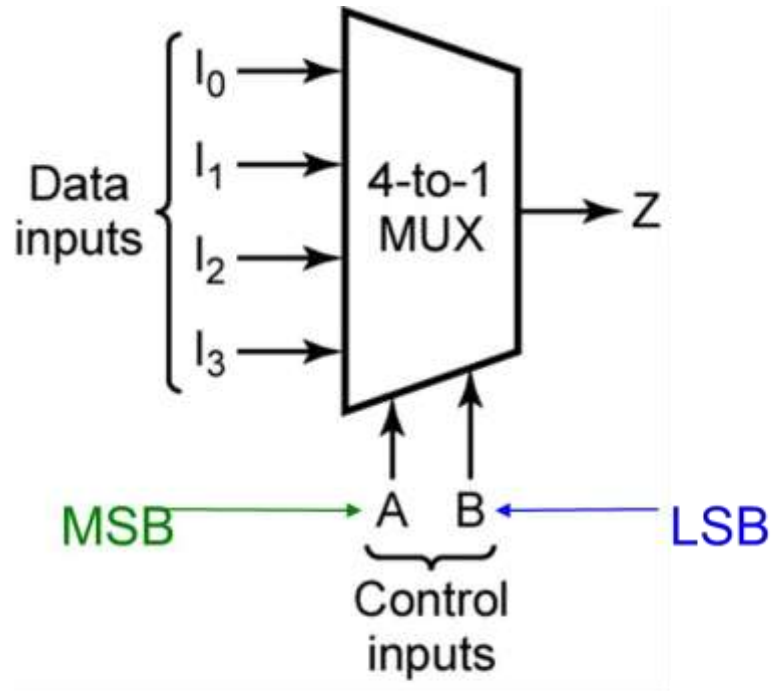
4-to-1 (2 select lines)

8-to-1 (3 select lines)

16-to-1 (4 select lines)



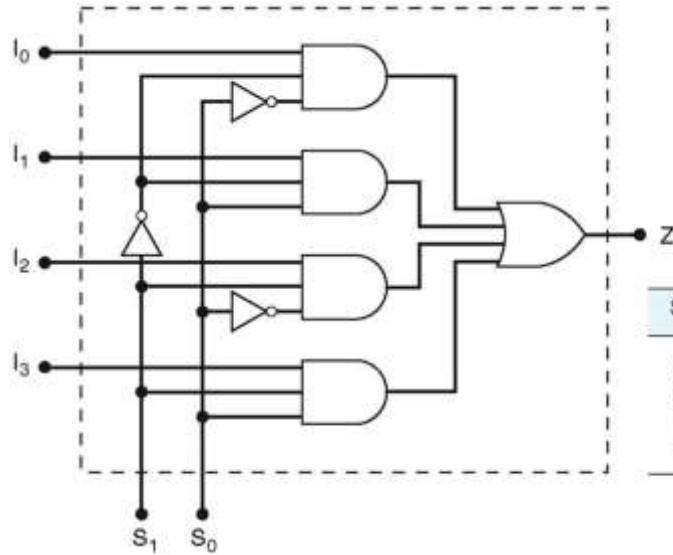




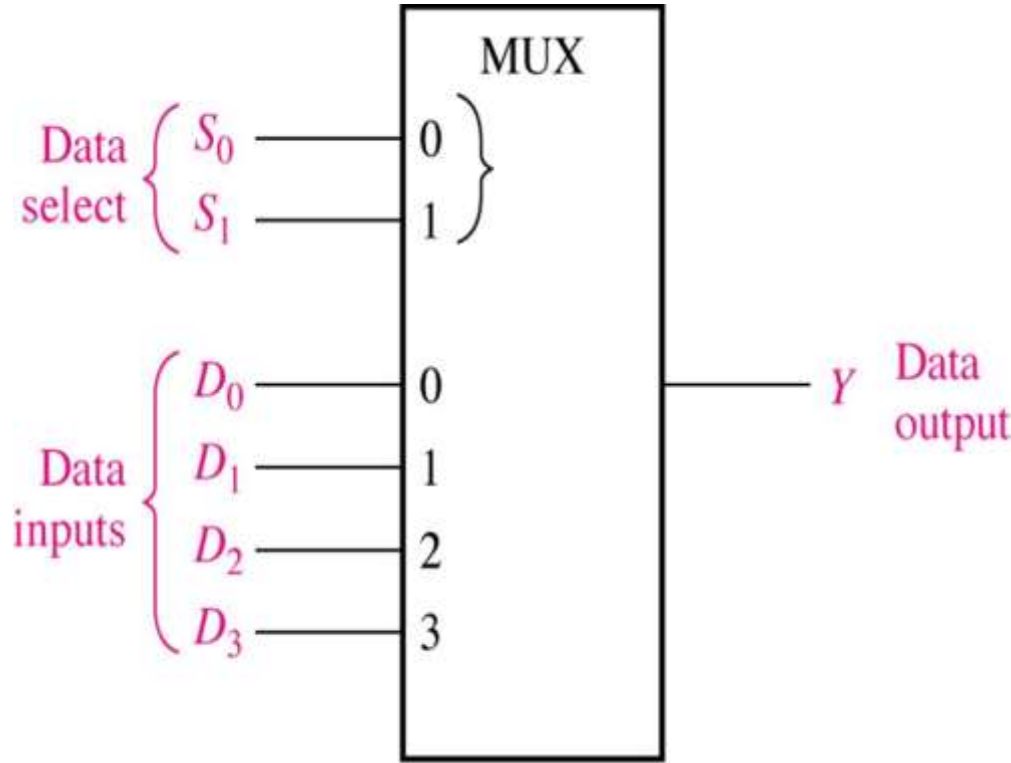
A	B	F
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Select lines are called as control lines

$$Z = A'.B'.I_0 + A'.B.I_1 + A.B'.I_2 + A.B.I_3$$



S_1	S_0	Output
0	0	$Z = I_0$
0	1	$Z = I_1$
1	0	$Z = I_2$
1	1	$Z = I_3$



DEMUX

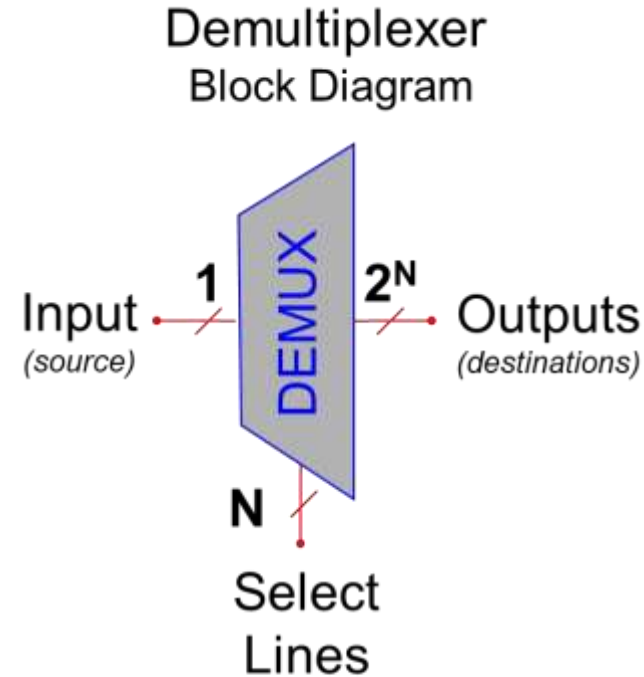
The select lines determine which output the input is connected to.

A DEMULTIPLEXER (DEMUX) basically reverses the multiplexing function. It takes data from one line and distributes them to a given number of output lines.

For this reason, the demultiplexers is also known as a data distributor.

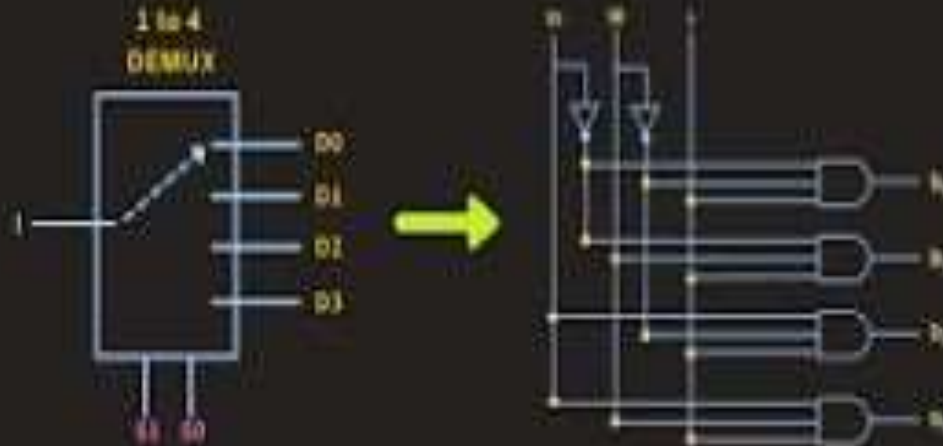
- DEMUX Types

- 1-to-2 (1 select line)
- 1-to-4 (2 select lines)
- 1-to-8 (3 select lines)
- 1-to-16 (4 select lines)





Demultiplexer



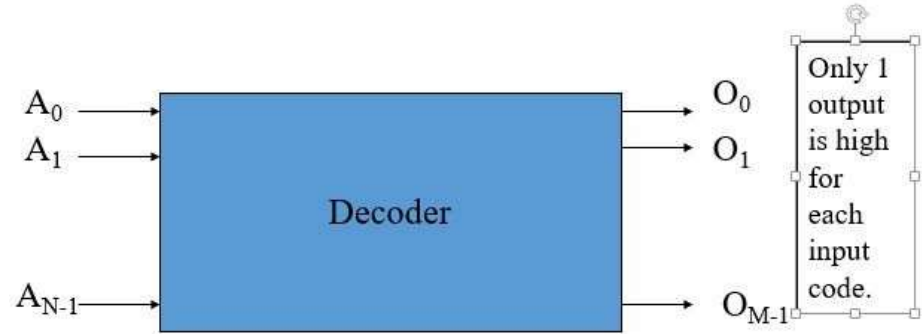
Input	Select Lines	Output Lines
I	$S_1 S_0$	$D_0 D_1 D_2 D_3$
I	0 0	1 0 0 0
I	0 1	0 1 0 0
I	1 0	0 0 1 0
I	1 1	0 0 0 1

Decoder

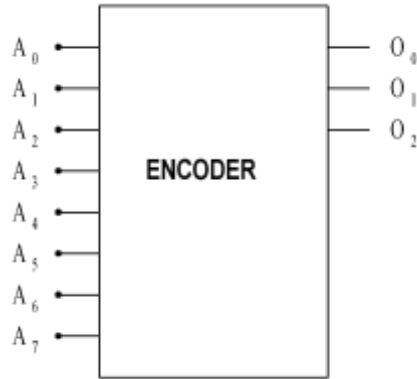
- A decoder is a logic circuit that accepts a **set of inputs that represents a binary number** and activates only the output that corresponds to that input number.

- Types

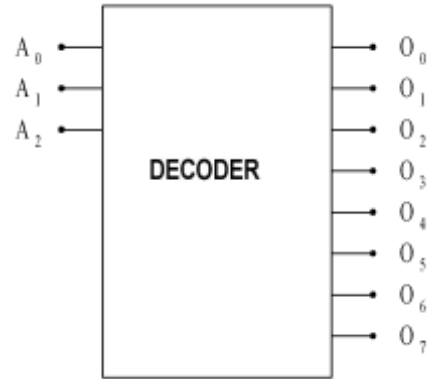
- 2 to 4 decoders
 - 3 to 8 decoders
 - 4 to 16 decoders
- Some decoders do not utilize all of the 2^N possible input codes, e.g., BCD-to-decimal decoder has a 4-bit input code and 10 output lines.



ENCODERS AND DECODERS



ONLY ONE INPUT
ACTIVATED AT A TIME
BINARY CODE
OUTPUT

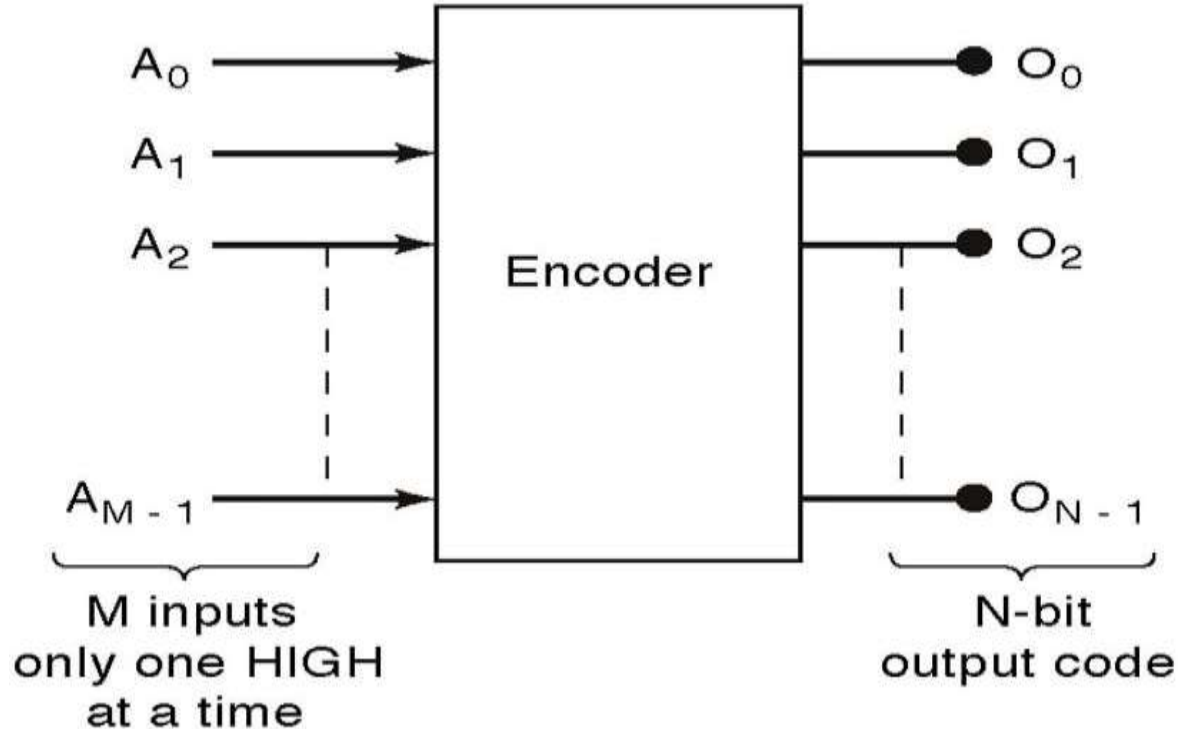


BINARY CODE INPUT
ONLY ONE OUTPUT
ACTIVATED AT A TIME

Enable input is used to control the operation of the
decoders

Encoders

- The opposite of the decoding process.
- An encoder has a number of input lines, only one of which is activated at a given time.
- Octal-to-binary encoder
- Decimal to BCD encoder



Encoder Circuit Design for octal to binary

- Example:
 - 8-3 Binary Encoder

Inputs								Outputs		
D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀	A ₂	A ₁	A ₀
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$A_0 = D_1 + D_3 + D_5 + D_7$$

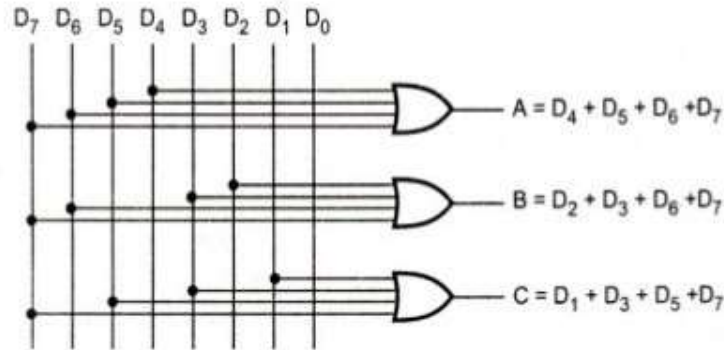
$$A_1 = D_2 + D_3 + D_6 + D_7$$

$$A_2 = D_4 + D_5 + D_6 + D_7$$

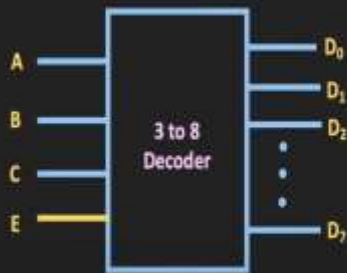
Circuit diagram for octal to binary



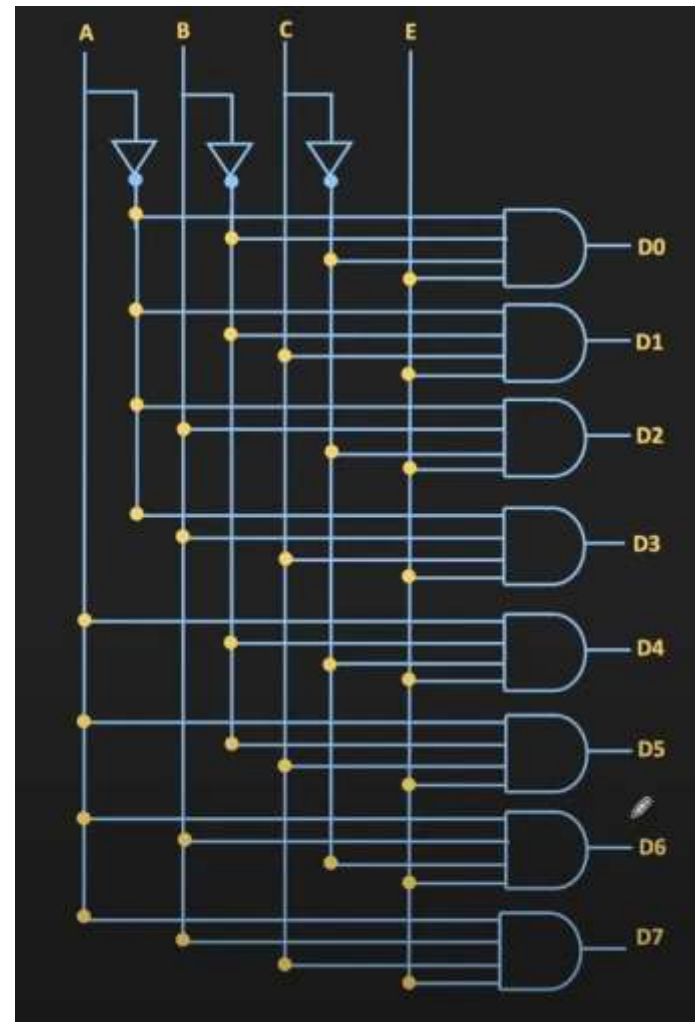
Octal to Binary Encoder



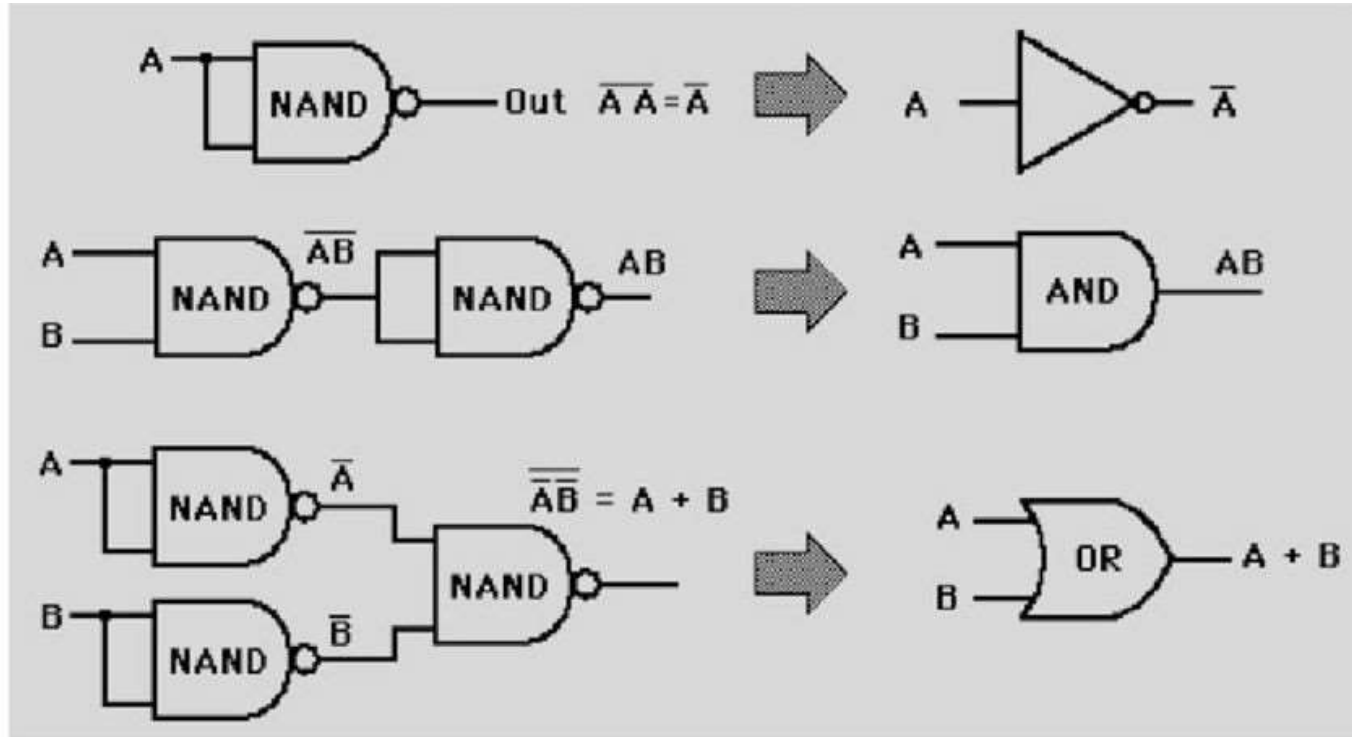
Decoder with Enable Input



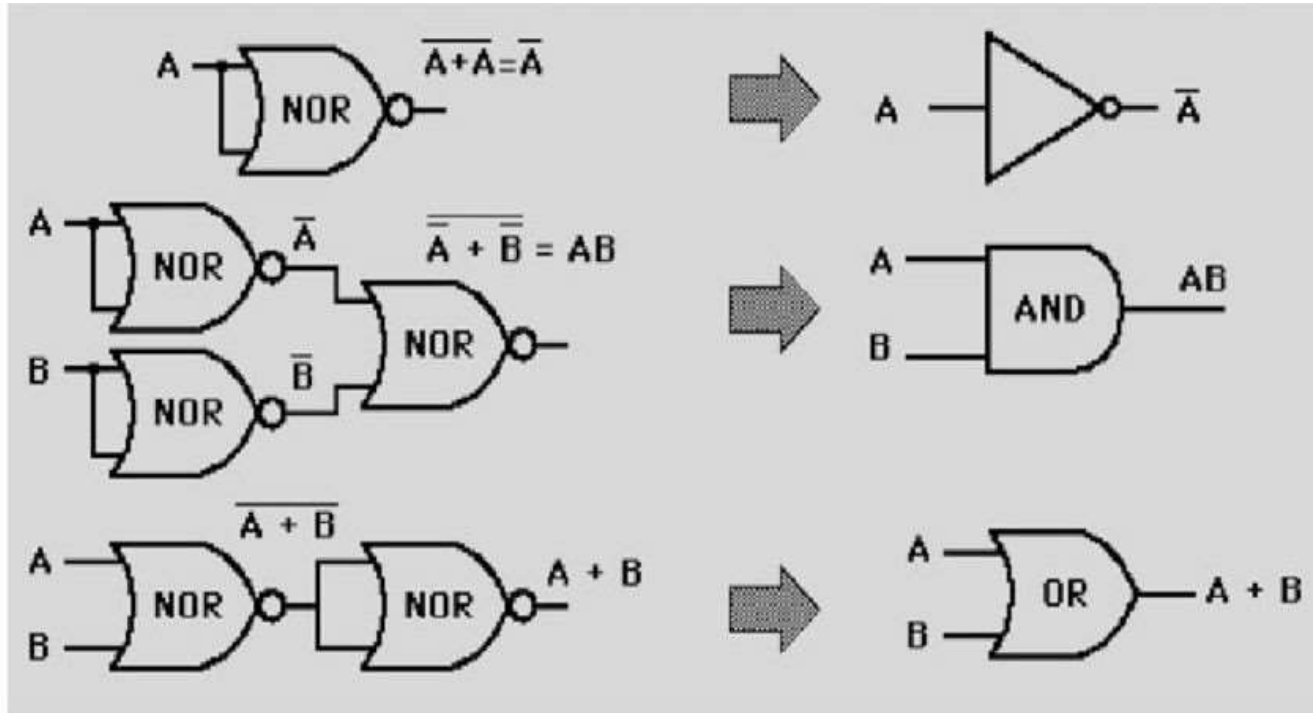
Inputs				Outputs							
EN	A	B	C	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	×	×	×	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0



Self learning topic -realisation of basic gates using NAND

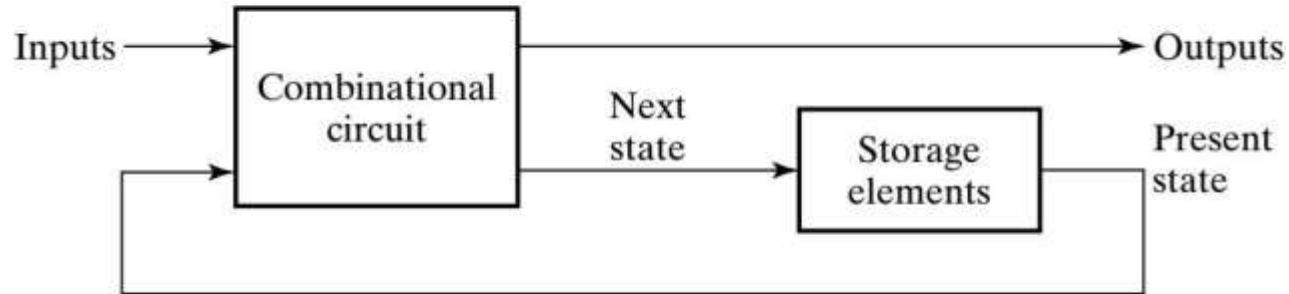


Self learning topic -realisation of basic gates using NOR



REALISE ANY CIRCUIT USING NAND GATE

SEQUENTIAL CIRCUITS



□ **FIGURE 1**
Block Diagram of a Sequential Circuit

- Flip-flops

The term "flip-flop" relates to the actual operation of the device, as it can be "flipped" to a logic set state or "flopped" back to the opposing logic reset state.

Types

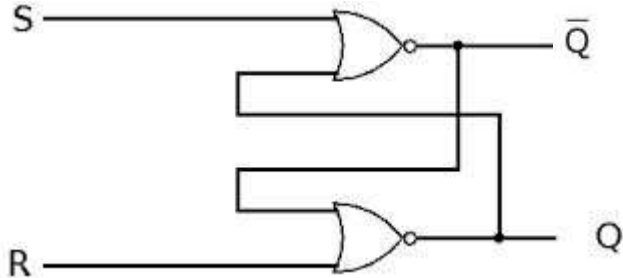
- The **SR ("set-reset")** Flipflop,
- The **D ("data" or "delay")** Flipflop,
- The **T ("toggle")** Flipflop, and
- **JK** Flip Flop.

SR FLIP FLOP

The SET input 'S' set the device or produce the output 1, and the RESET input 'R' reset the device or produce the output 0. The SET and RESET inputs are labeled as **S** and **R**, respectively.

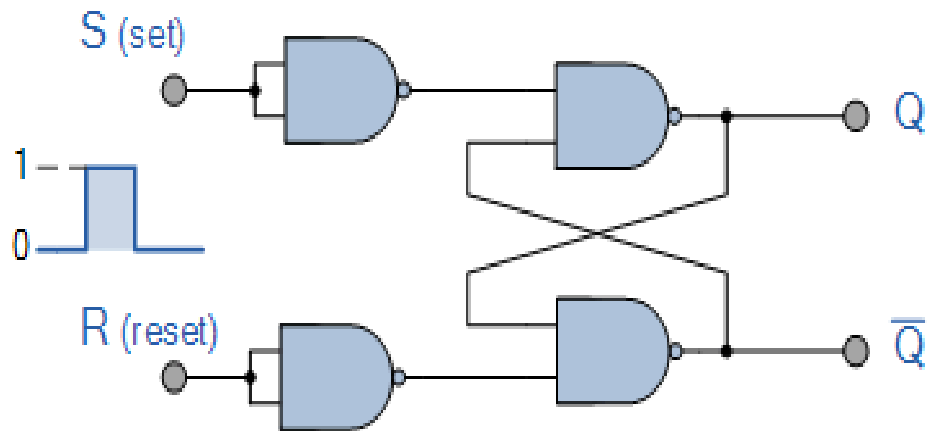
The SR flip flop stands for "Set-Reset" flip flop. The reset input is used to get back the flip flop to its original state from the current state with an output 'Q'. This output depends on the set and reset conditions, which is either at the logic level "0" or "1".

SR NOR LATCH



INPUT		OUTPUT
S	R	Q_{n+1}
0	0	Q_n
1	0	1
0	1	0
1	1	Invalid

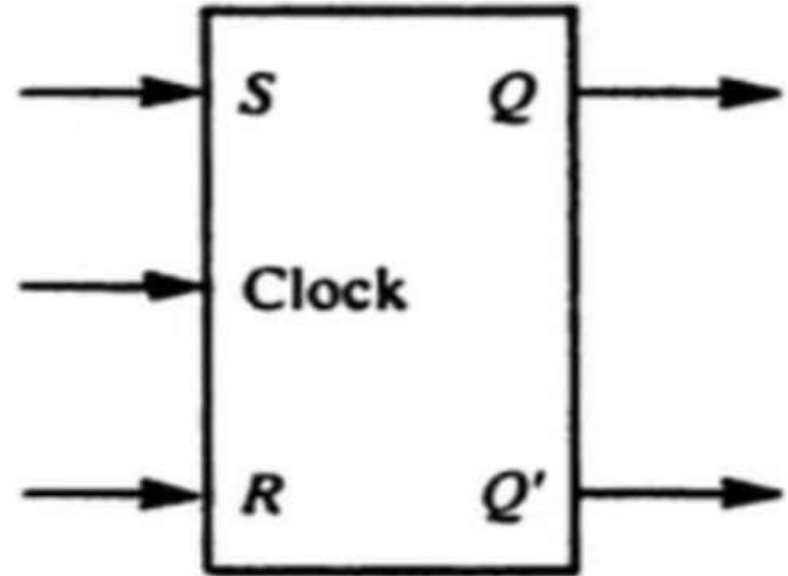
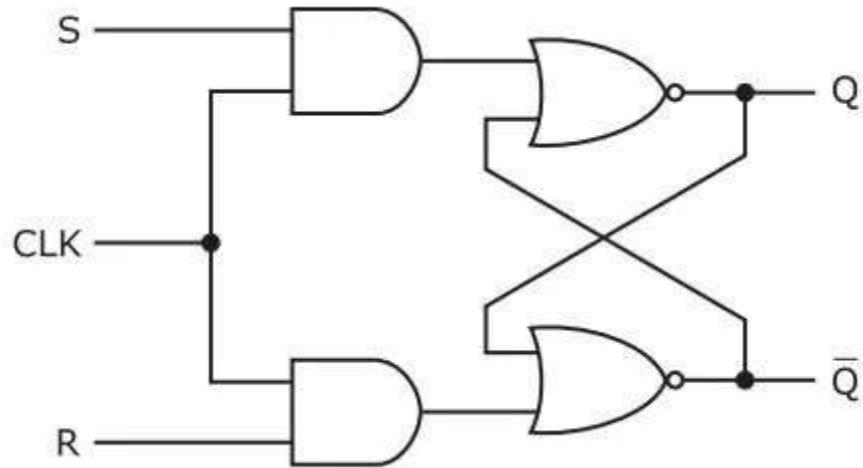
Positive NAND Gate SR Flip-flop

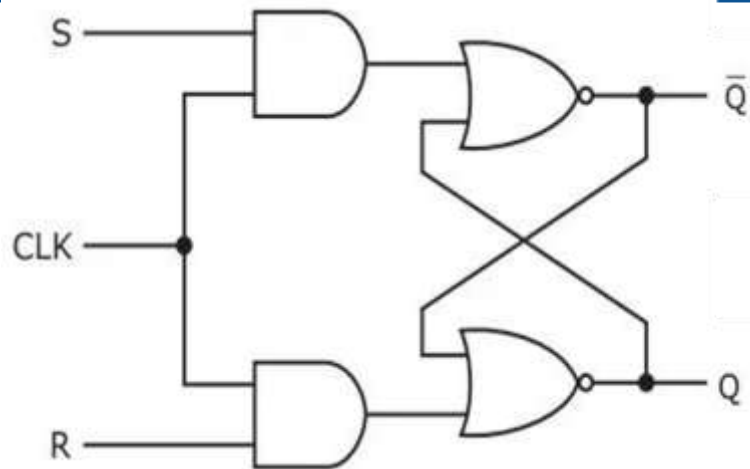


NAND Circuit

S	R	Q	$\bar{Q}$
0	0	No change	
0	1	0	1
1	0	1	0
1	1	X	X
(Invalid)			

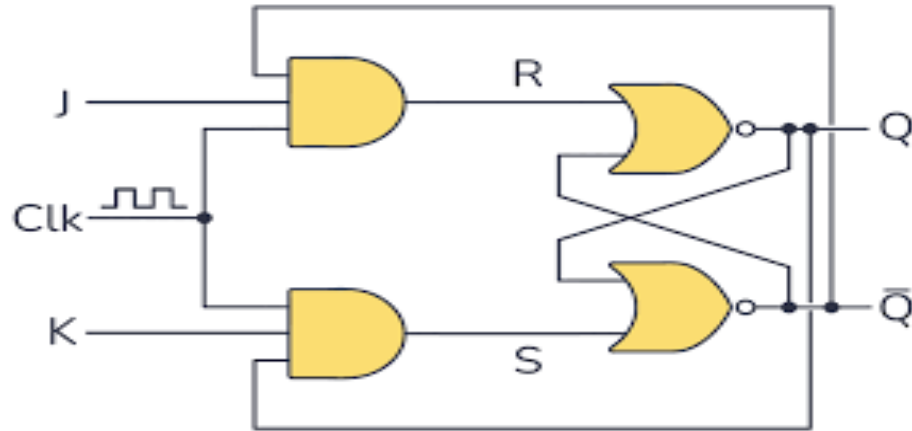
SR FLIP FLOP





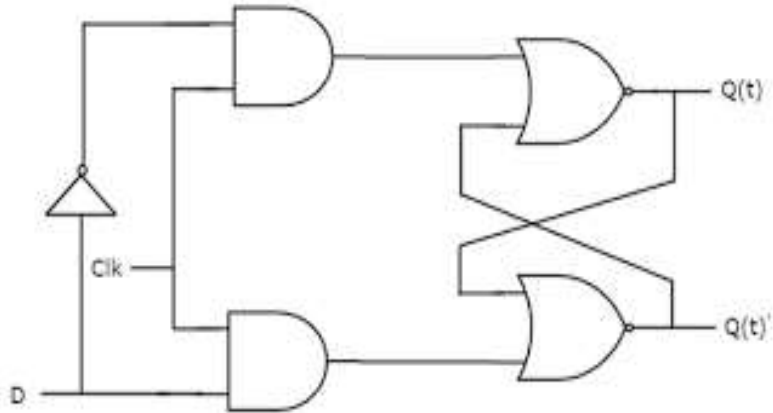
CP	S	R	Q _{n+1}	State
1	0	1	0	Reset
1	1	0	1	Set
1	1	1	X	Invalid
1	0	0	Q _n	No change

JK FLIP FLOP



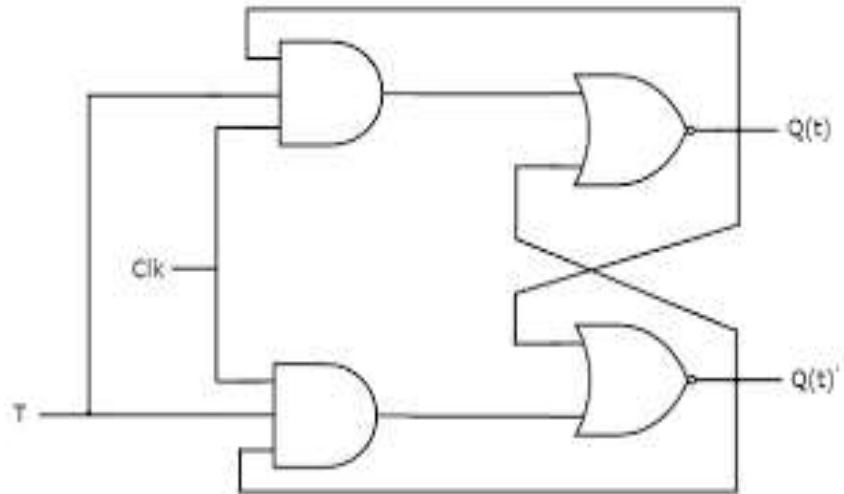
INPUT		OUTPUT
J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Q_n'

D FLIP FLOP



D INPUT	Q OUTPUT
0	0
1	1

T FLIP FLOP



T INPUT	Q_{n+1}
0	Q_n
1	Q_n'

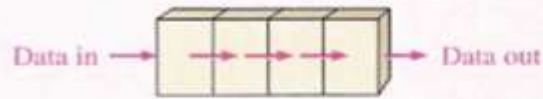
REGISTERS

Designed by using FF

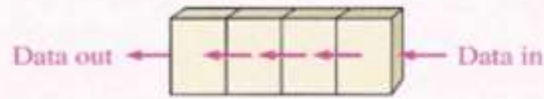
- FF is capable of storing 1 bit (0/1)
- FFs are grouped together to design Register
- To design n bit register, we need n FF
- Which type of FF???
- D FF
- With added combinational gates, the register can perform data-processing tasks.
- A counter is a register that goes through a predetermined sequence of states upon the application of clock pulses.

Data formats in registers

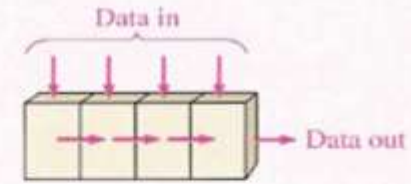
- Serial form
- Parallel form



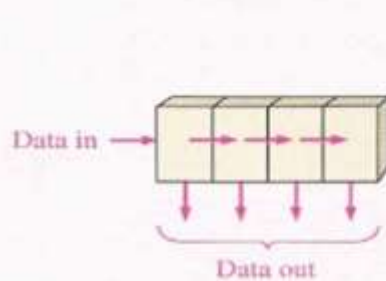
(a) Serial in/shift right/serial out



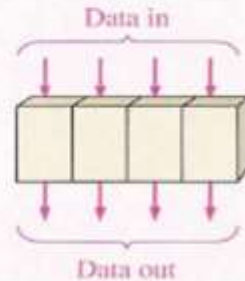
(b) Serial in/shift left/serial out



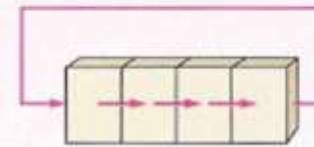
(c) Parallel in/serial out



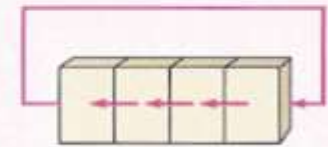
(d) Serial in/parallel out



(e) Parallel in/parallel out



(f) Rotate right



(g) Rotate left

Classification of Registers

- Based on inputs and outputs
 - SISO
 - SIPO
 - PISO
 - PIPO
- Depending on application
 - Shift registers
 - Storage registers

Shift Register

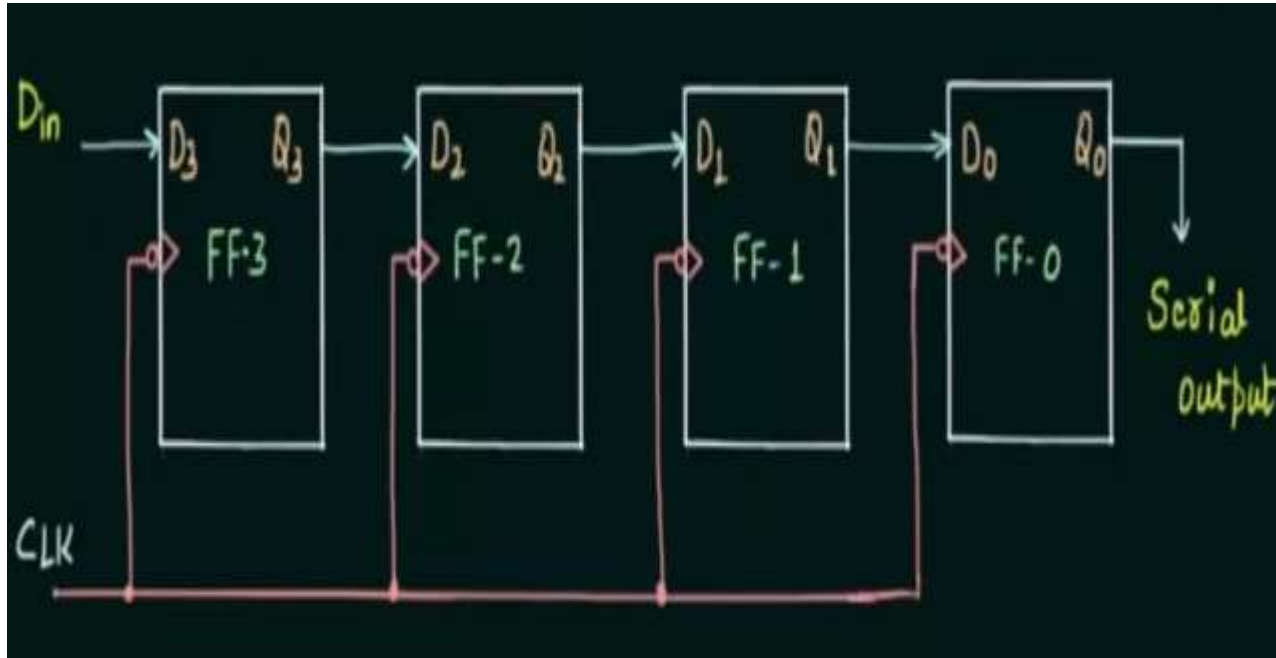
- The **Shift Register** is used for the storage and the transfer of data in the form of binary numbers.
- The shift register loads the data present on its inputs and then moves or “shifts” it to its output once every clock cycle, hence the name “shift register”.
- **Applications**
 - *Shift Registers* are used for data storage or for the movement of data and are therefore commonly used inside calculators or computers to store data such as two binary numbers before they are added together,
 - *Shift Registers* are used to convert the data from either a serial to parallel or parallel to serial format

Modes of operation of a shift register

- **Serial-in to Parallel-out (SIPO)** - the register is loaded with serial data, one bit at a time, with the stored data being available at the output in parallel form.
- **Serial-in to Serial-out (SISO)** - the data is shifted serially “IN” and “OUT” of the register, one bit at a time in either a left or right direction under clock control.
- **Parallel-in to Serial-out (PISO)** - the parallel data is loaded into the register simultaneously and is shifted out of the register serially one bit at a time under clock control.
- **Parallel-in to Parallel-out (PIPO)** - the parallel data is loaded simultaneously into the register, and transferred together to their respective outputs by the same clock pulse.

SISO shift Register

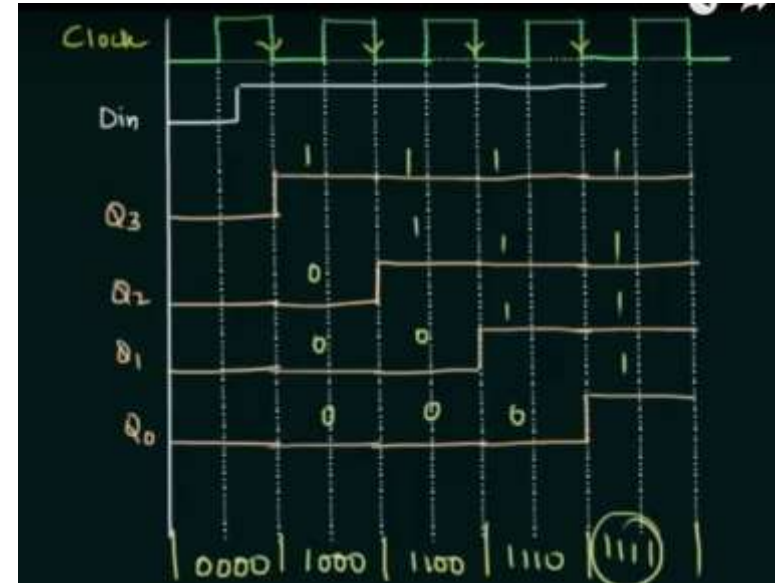
1111 is the data,, start
from LSB,from D3



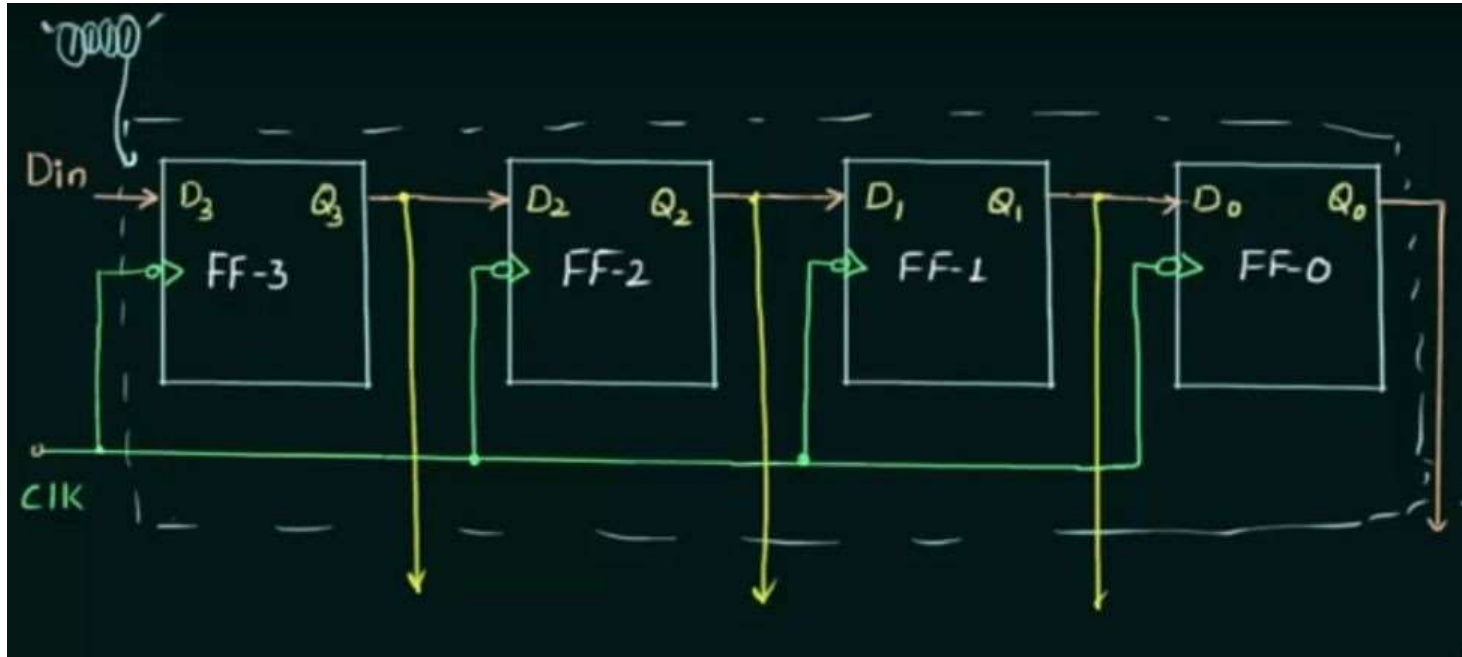
4 clock pulses to store 4 bit and 4 clock pulses to retrieve the 4 bit data

To store 1111, SISO shift register....(4 clock pulses)

CLK	Q_3	Q_2	Q_1	Q_0
Initially	0	0	0	0
↓	1	0	0	0
↓	1	1	0	0
↓	1	1	1	0
↓	1	1	1	1

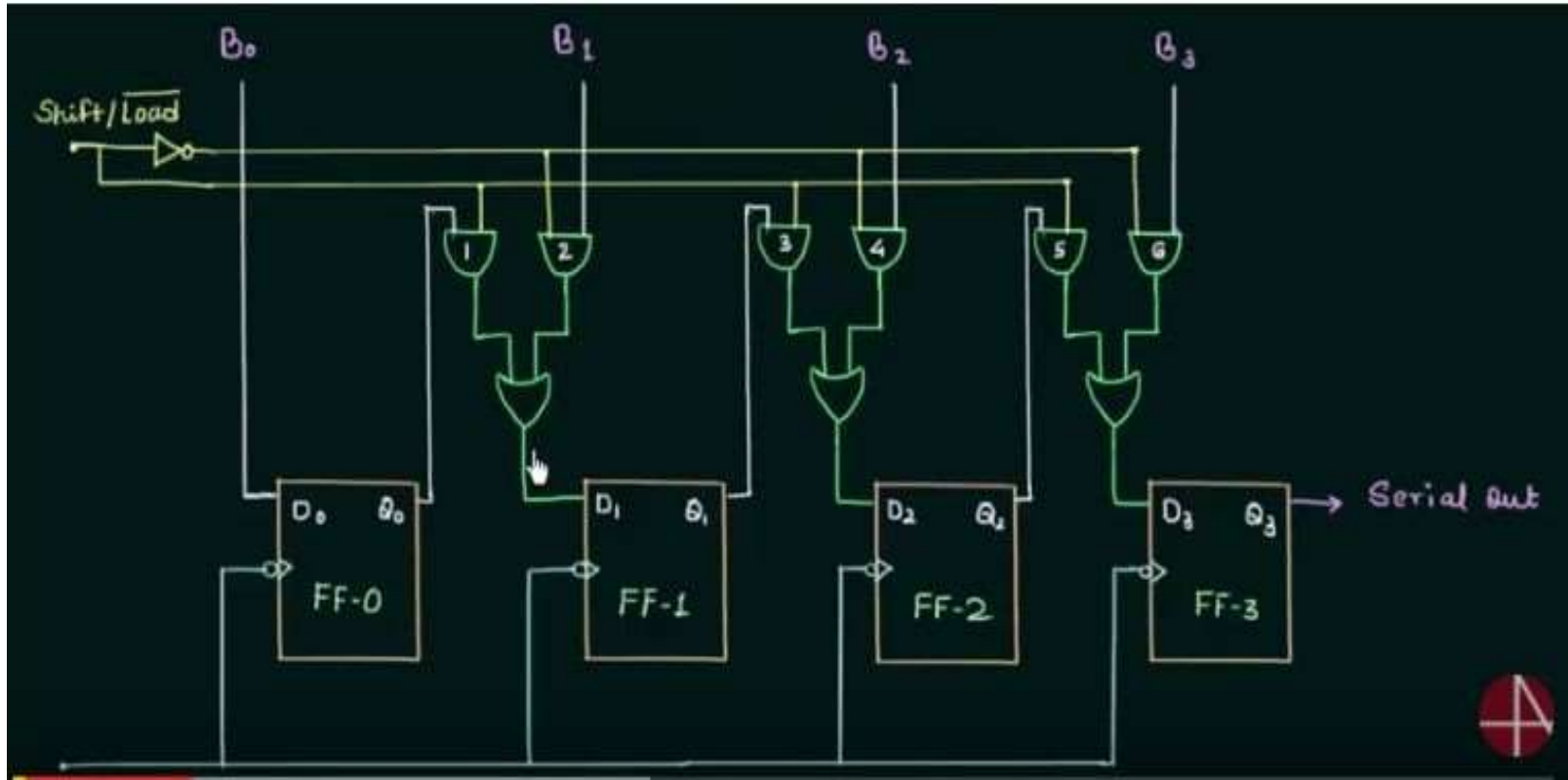


SIPO shift register



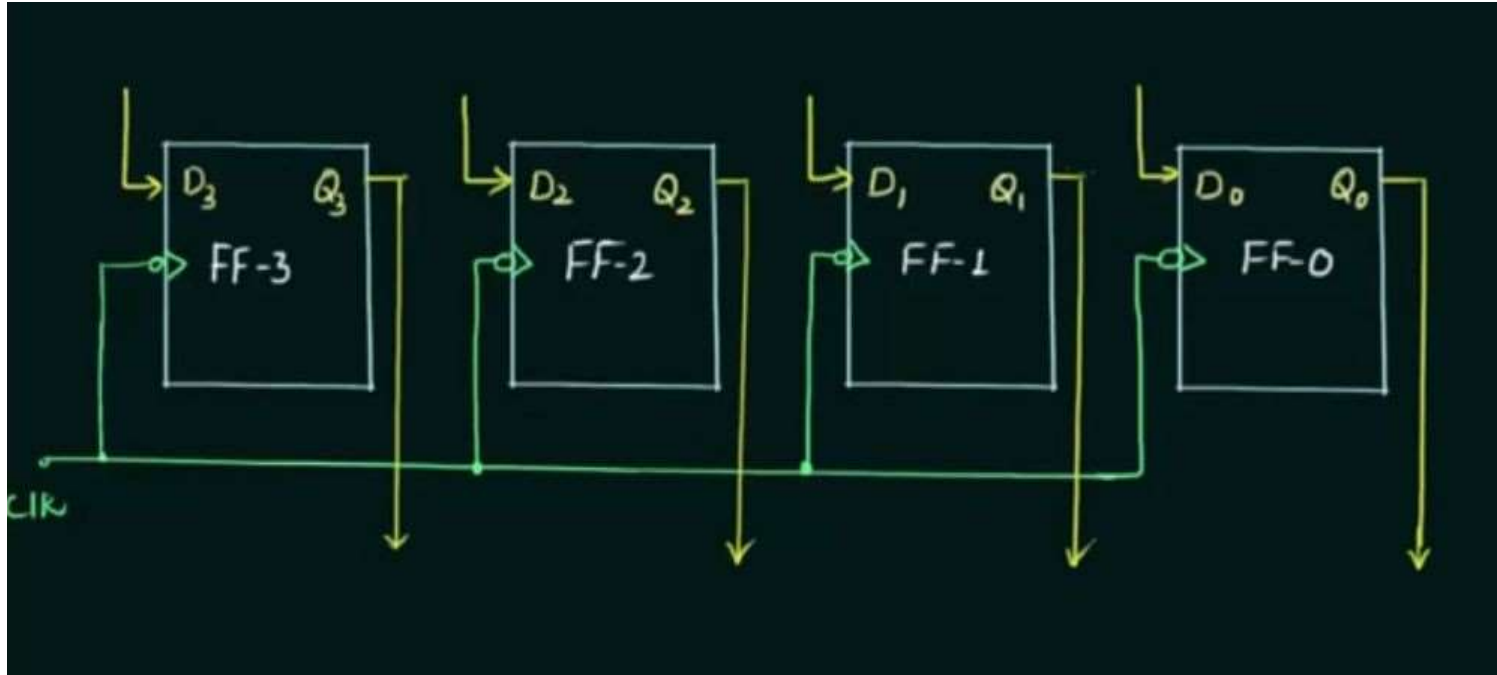
4 clock pulses to store 4 bit data ; 1 clock pulse to retrieve the 4 bit data

PISO



PIPO shift register

1 clock pulse to load 4 bit and 1 clock pulse to retrieve 4 bit data



This shift register is called as storage register or buffer register....

Register transfer and micro operations

Combinational and sequential circuits can be used to create simple digital systems.

These are the low-level building blocks of a digital computer.

Microoperations.

The operations on the data in registers are called microoperations.

Shift

Load

Clear

Increment

...

Register Transfer Language

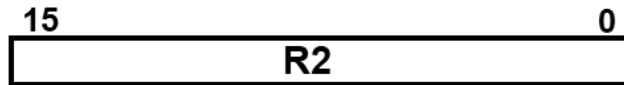
Register transfer language

- A symbolic language
- A convenient tool for describing the internal organization of digital computers
- Can also be used to facilitate the design process of digital systems.

DESIGNATION OF REGISTERS

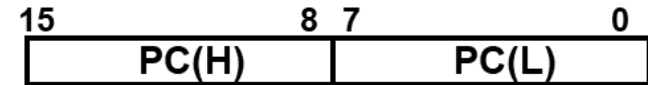
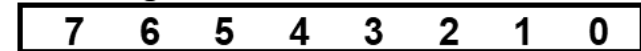
- Designation of a register
 - a register
 - portion of a register
 - a bit of a register
- Common ways of drawing the block diagram of a register

Register



Numbering of bits

Showing individual bits



Subfields

REGISTER TRANSFER

A register transfer such as

$R3 \leftarrow R5$

Implies that the digital system has

the data lines from the source register (R5) to the destination register (R3)

Parallel load in the destination register (R3)

Control lines to perform the action

CONTROL FUNCTIONS

Often actions need to only occur if a certain condition is true

This is similar to an “if” statement in a programming language

In digital systems, this is often done via a control signal, called a control function

If the signal is 1, the action takes place This is represented as:

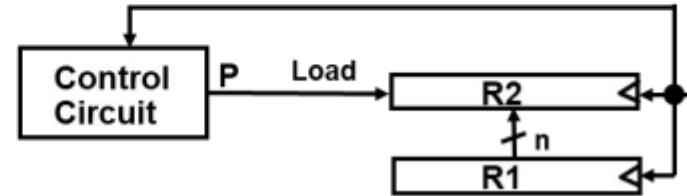
P: $R2 \leftarrow R1$

Which means “if $P = 1$, then load the contents of register $R1$ into register $R2$ ”, i.e., if $(P = 1)$ then $(R2 \leftarrow R1)$

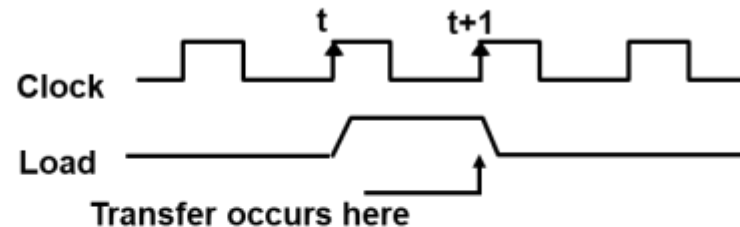
Implementation of controlled transfer

$$P: R2 \leftarrow R1$$
HARDWARE
IMPLEMENTATION
OF CONTROLLED
TRANSFERS

Block diagram



Timing diagram



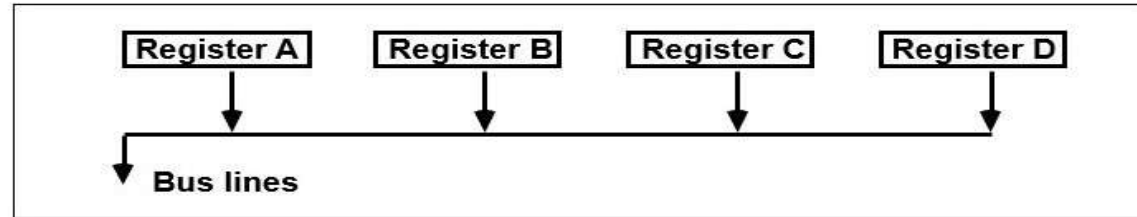
- The same clock controls the circuits that generate the control function and the destination register
- Registers are assumed to use *positive-edge-triggered* flip-flops

BASIC SYMBOLS FOR REGISTER TRANSFERS

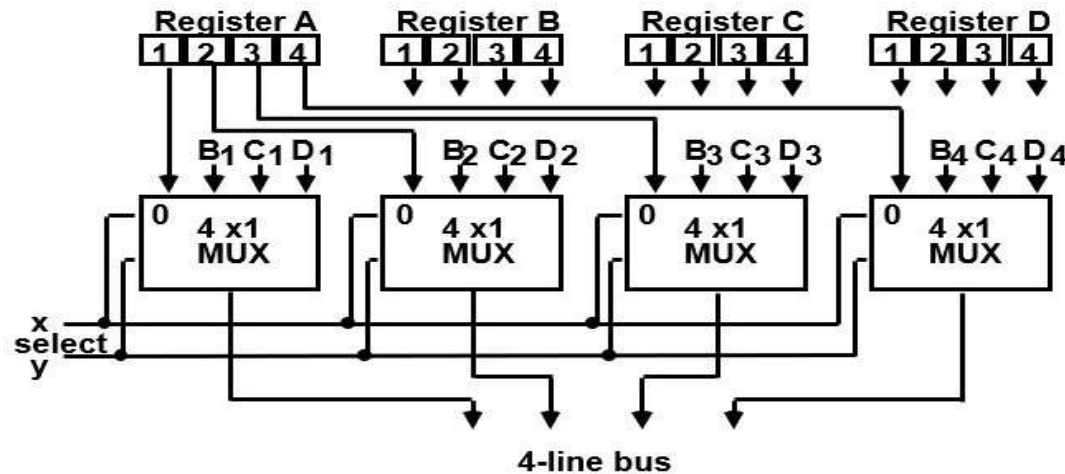
Symbols	Description	Examples
Capital letters & numerals	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	R2(0-7), R2(L)
Arrow ←	Denotes transfer of information	R2 ← R1
Colon :	Denotes termination of control function	P:
Comma ,	Separates two micro-operations	A ← B, B ← A

Bus is a path(of a group of wires) over which information is transferred, from any of several sources to any of several destinations

From a register to bus: $BUS \leftarrow R$

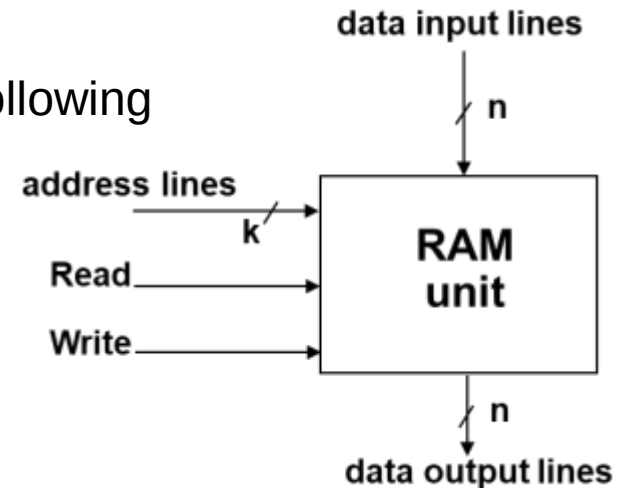


**BUS AND
BUS
TRANSFER**



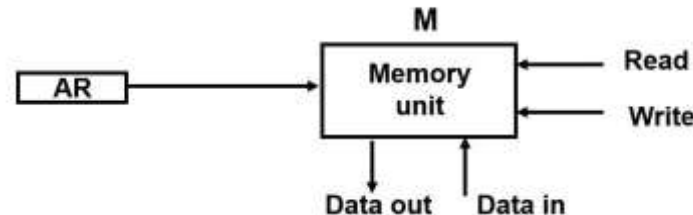
MEMORY

- Memory (RAM) can be thought as a sequential circuits containing some number of registers
- These registers hold the *words* of memory
- Each of the r registers is indicated by an *address*
- These addresses range from 0 to $r-1$
- Each register (word) can hold n bits of data
- Assume the RAM contains $r = 2^k$ words. It needs the following
 - n data input lines
 - n data output lines
 - k address lines
 - A Read control line
 - A Write control line



MEMORY TRANSFER

- Collectively, the memory is viewed at the register level as a device, M.
- Since it contains multiple locations, we must specify which **address** in memory we will be using
- This is done by **indexing memory references**
- Memory is usually accessed in computer systems by putting the desired address in a special register, the **Memory Address Register (MAR, or AR)**
- When memory is accessed, the contents of the MAR get sent to the memory unit's address lines



MEMORY READ

- To read a value from a location in memory and load it into a register, the register transfer language notation looks like this:
 - $R1 \leftarrow M[MAR]$
- This causes the following to occur
 - The contents of the MAR get sent to the memory address lines
 - A Read (= 1) gets sent to the memory unit
 - The contents of the specified address are put on the memory's output data lines
 - These get sent over the bus to be loaded into register R1

Memory write

- To write a value from a register to a location in memory looks like this in register transfer language:
 - $M[MAR] \leftarrow R1$
- This causes the following to occur
 - The contents of the MAR get sent to the memory address lines
 - A Write (= 1) gets sent to the memory unit
 - The values in register R1 get sent over the bus to the data input lines of the memory
 - The values get loaded into the specified address in the memory

SUMMARY OF REGISTER TRANSFER MICRO OPERATIONS

$A \leftarrow B$	Transfer content of reg. B into reg. A
$AR \leftarrow DR(AD)$	Transfer content of AD portion of reg. DR into reg. AR
$A \leftarrow \text{constant}$	Transfer a binary constant into reg. A
$ABUS \leftarrow R1,$ $R2 \leftarrow ABUS$	Transfer content of R1 into bus A and, at the same time, transfer content of bus A into R2
AR	Address register
DR	Data register
M[R]	Memory word specified by reg. R
M	Equivalent to M[AR]
$DR \leftarrow M$	Memory <i>read</i> operation: transfers content of memory word specified by AR into DR
$M \leftarrow DR$	Memory <i>write</i> operation: transfers content of DR into memory word specified by AR

LOGIC MICROOPERATIONS

- **Specify binary operations on the strings of bits in registers**
 - Logic microoperations are bit-wise operations, i.e., they work on the individual bits of data
 - useful for bit manipulations on binary data
 - useful for making logical decisions based on the bit value
- **There are, in principle, 16 different logic functions that can be defined over two binary input variables**

A	B	F ₀	F ₁	F ₂	...	F ₁₃	F ₁₄	F ₁₅
0	0	0	0	0	...	1	1	1
0	1	0	0	0	...	1	1	1
1	0	0	0	1	...	0	1	1
1	1	0	1	0	...	1	0	1

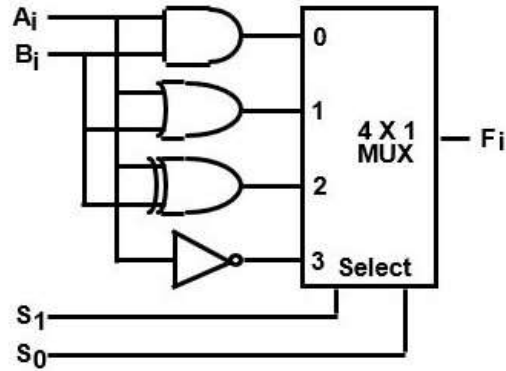
- However, most systems only implement four of these

AND ($\wedge$), OR ($\vee$), XOR ($\oplus$), Complement/NOT

- The others can be created from combination of these

- **List of Logic Microoperations**
 - 16 different logic operations with 2 binary vars.
 - n binary vars $\rightarrow 2^{2^n}$ functions
- **Truth tables for 16 functions of 2 variables and the corresponding 16 logic micro-operations**

x	0 0 1 1	<i>Boolean Function</i>	<i>Micro- Operations</i>	<i>Name</i>
y	0 1 0 1			
	0 0 0 0	$F_0 = 0$	$F \leftarrow 0$	Clear
	0 0 0 1	$F_1 = xy$	$F \leftarrow A \wedge B$	AND
	0 0 1 0	$F_2 = xy'$	$F \leftarrow A \wedge B'$	
	0 0 1 1	$F_3 = x$	$F \leftarrow A$	Transfer A
	0 1 0 0	$F_4 = x'y$	$F \leftarrow A' \wedge B$	
	0 1 0 1	$F_5 = y$	$F \leftarrow B$	Transfer B
	0 1 1 0	$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
	0 1 1 1	$F_7 = x + y$	$F \leftarrow A \vee B$	OR
	1 0 0 0	$F_8 = (x + y)'$	$F \leftarrow (A \vee B)'$	NOR
	1 0 0 1	$F_9 = (x \oplus y)'$	$F \leftarrow (A \oplus B)'$	Exclusive-NOR
	1 0 1 0	$F_{10} = y'$	$F \leftarrow B'$	Complement B
	1 0 1 1	$F_{11} = x + y'$	$F \leftarrow A \vee B$	
	1 1 0 0	$F_{12} = x'$	$F \leftarrow A'$	Complement A
	1 1 0 1	$F_{13} = x' + y$	$F \leftarrow A' \vee B$	
	1 1 1 0	$F_{14} = (xy)'$	$F \leftarrow (A \wedge B)'$	NAND
	1 1 1 1	$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's



Function table

S_1	S_0	Output	μ -operation
0	0	$F = A \wedge B$	AND
0	1	$F = A \vee B$	OR
1	0	$F = A \oplus B$	XOR
1	1	$F = A'$	Complement

APPLICATIONS OF LOGIC MICROOPERATIONS

- Logic microoperations can be used to manipulate individual bits or a portions of a word in a register
- Consider the data in a register A. In another register, B, is bit data that will be used to modify the contents of A

- | | |
|------------------------|--------------------------------|
| – Selective-set | $A \leftarrow A + B$ |
| – Selective-complement | $A \leftarrow A \oplus B$ |
| – Selective-clear | $A \leftarrow A \cdot B'$ |
| – Mask (Delete) | $A \leftarrow A \cdot B$ |
| – Clear | $A \leftarrow A \oplus B$ |
| – Insert | $A \leftarrow (A \cdot B) + C$ |
| – Compare | $A \leftarrow A \oplus B$ |
| – ... | |

SELECTIVE SET

- In a selective set operation, the bit pattern in B is used to **set** certain bits in A

$$\begin{array}{rcl} 1\ 1\ 0\ 0 & A_t & \\ 1\ 0\ 1\ 0 & B & \\ \hline 1\ 1\ 1\ 0 & A_{t+1} & (A \leftarrow A + B) \end{array}$$

- If a bit in B is set to 1, that same position in A gets set to 1, otherwise that bit in A keeps its previous value

SELECTIVE COMPLEMENT

- In a selective complement operation, the bit pattern in B is used to *complement* certain bits in A

$$\begin{array}{rcl} 1100 & A_t & \\ 1010 & B & \\ \hline 0110 & A_{t+1} & (A \leftarrow A \oplus B) \end{array}$$

- If a bit in B is set to 1, that same position in A gets complemented from its original value, otherwise it is unchanged

- In a selective clear operation, the bit pattern in B is used to *clear* certain bits in A

$$\begin{array}{rcl} 1100 & A_t & \\ 1010 & B & \\ \hline 0100 & A_{t+1} & (A \leftarrow A \cdot B') \end{array}$$

- If a bit in B is set to 1, that same position in A gets set to 0, otherwise it is unchanged

MASK OPERATION

- In a mask operation, the bit pattern in B is used to *clear* certain bits in A

$$\begin{array}{rcl} 1100 & A_t & \\ 1010 & B & \\ \hline 1000 & A_{t+1} & (A \leftarrow A \cdot B) \end{array}$$

- If a bit in B is set to 0, that same position in A gets set to 0, otherwise it is unchanged

CLEAR OPERATION

- In a clear operation, if the bits in the same position in A and B are the same, they are cleared in A, otherwise they are set in A

$$\begin{array}{rcl} 1100 & A_t & \\ 1010 & B & \\ \hline 0110 & A_{t+1} & (A \leftarrow A \oplus B) \end{array}$$

INSERT OPERATION

- An insert operation is used to introduce a specific bit pattern into A register, leaving the other bit positions unchanged
- This is done as
 - A mask operation to clear the desired bit positions, followed by
 - An OR operation to introduce the new bits into the desired positions
 - Example
 - » Suppose you wanted to introduce 1010 into the low order four bits of A:

1101 1000 1011 0001 A (Original)

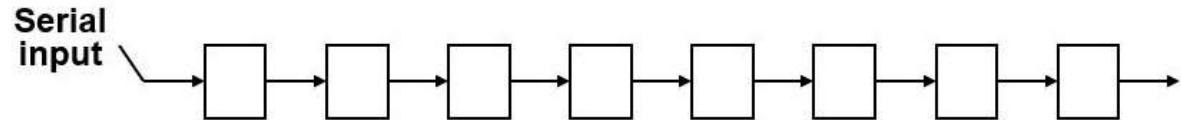
1101 1000 1011 1010 A (Desired)

» 1101 1000 1011 0001	A (Original)
1111 1111 1111 0000	Mask
1101 1000 1011 0000	A (Intermediate)
0000 0000 0000 1010	Added bits
1101 1000 1011 1010	A (Desired)

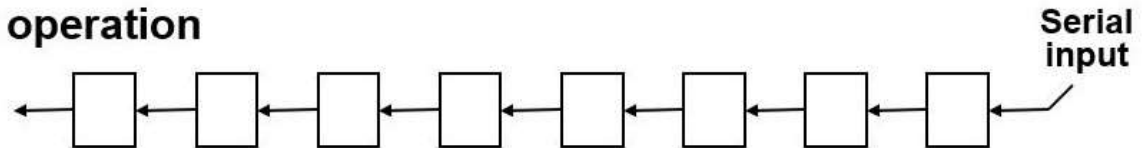
SHIFT MICRO OPERATIONS

- There are three types of shifts
 - *Logical shift*
 - *Circular shift*
 - *Arithmetic shift*
- What differentiates them is the information that goes into the serial input

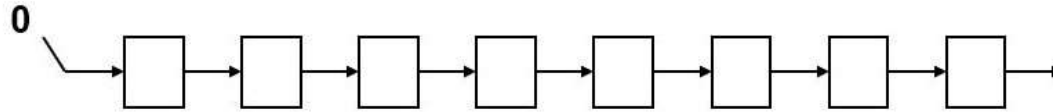
- A right shift operation



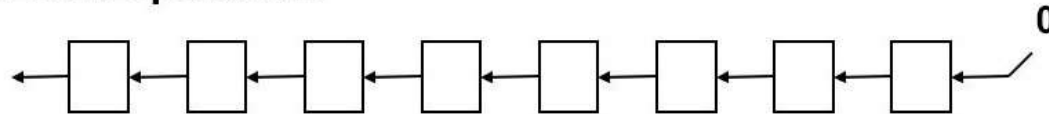
- A left shift operation



- In a logical shift the serial input to the shift is a 0.
- A right logical shift operation:



- A left logical shift operation:

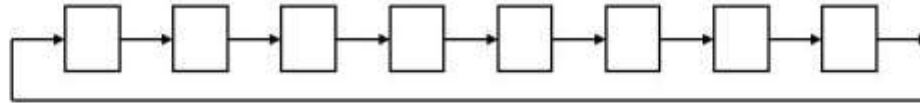


- In a Register Transfer Language, the following notation is used
 - *shl* for a logical shift left
 - *shr* for a logical shift right
 - Examples:
 - » $R2 \leftarrow shr\ R2$
 - » $R3 \leftarrow shl\ R3$

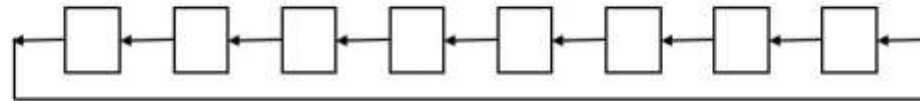
CIRCULAR SHIFT

- In a circular shift the serial input is the bit that is shifted out of the other end of the register.

- A right circular shift operation:



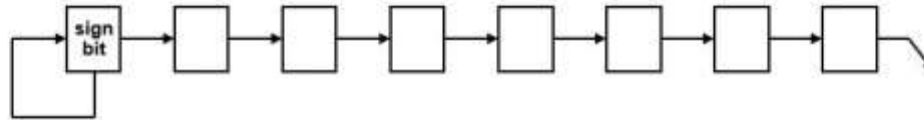
- A left circular shift operation:



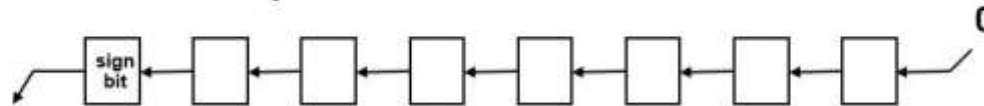
- In a RTL, the following notation is used
 - *cil* for a circular shift left
 - *cir* for a circular shift right
 - Examples:
 - » $R2 \leftarrow cir\ R2$
 - » $R3 \leftarrow cil\ R3$

ARITHMETIC SHIFT

- An arithmetic shift is meant for signed binary numbers (integer)
- An arithmetic left shift **multiplies** a signed number **by two**
- An arithmetic right shift **divides** a signed number **by two**
- The main distinction of an arithmetic shift is that it must keep the sign of the number the same as it performs the multiplication or division
- A right arithmetic shift operation:

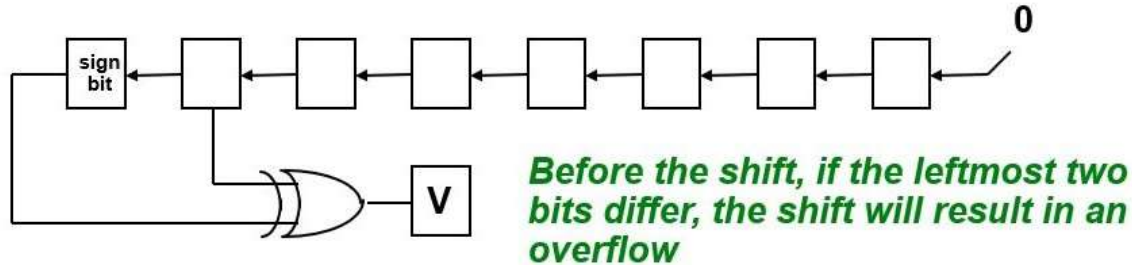


- A left arithmetic shift operation:

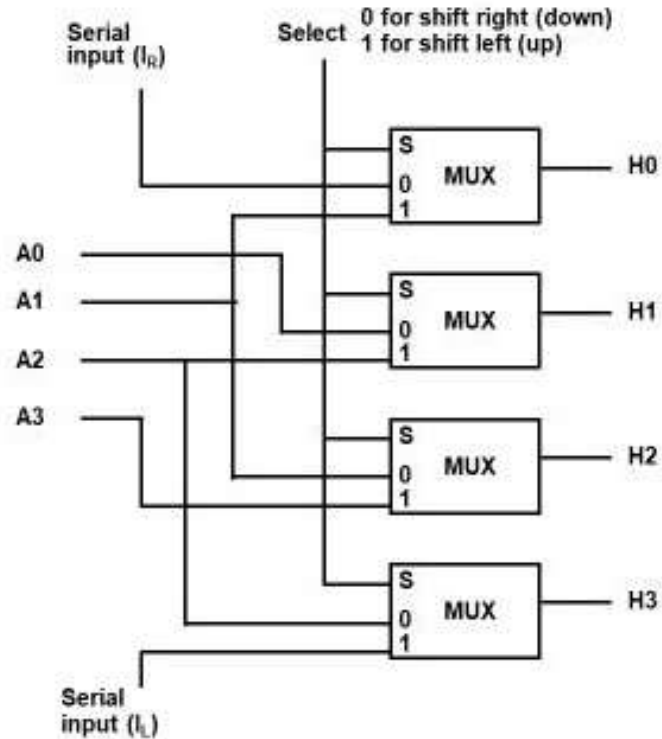


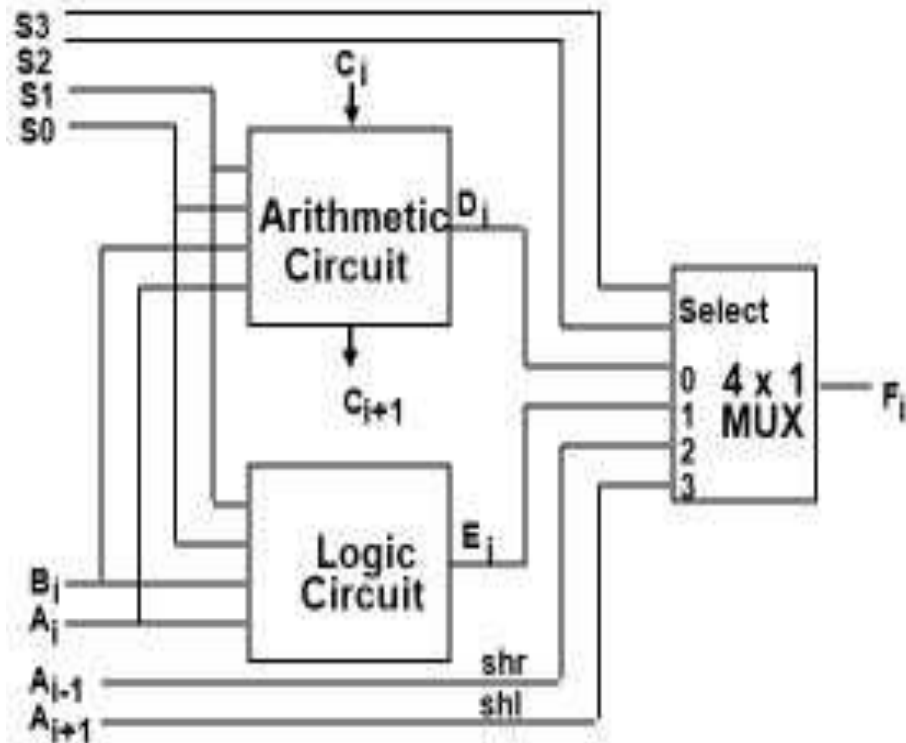
ARITHMETIC SHIFT

- An left arithmetic shift operation must be checked for the **overflow**



- In a RTL, the following notation is used
 - *ashl* for an arithmetic shift left
 - *ashr* for an arithmetic shift right
 - Examples:
 - » $R2 \leftarrow ashr\ R2$
 - » $R3 \leftarrow ashl\ R3$





S3	S2	S1	S0	Cin	Operation	Function
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + B'$	Subtract with borrow
0	0	1	0	1	$F = A + B' + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	X	$F = A \wedge B$	AND
0	1	0	1	X	$F = A \vee B$	OR
0	1	1	0	X	$F = A \oplus B$	XOR
0	1	1	1	X	$F = A'$	Complement A
1	0	X	X	X	$F = \text{shr } A$	Shift right A into F
1	1	X	X	X	$F = \text{shl } A$	Shift left A into F